

LLM-assisted Java

 LLM-native development



airhacks.industries

**"It's not work if you like it"
...so I never worked. #java**



**youtube.com/
@bienadam**



[/@bienadam/shorts](https://www.youtube.com/@bienadam/shorts)

#360 Java, LangChain4J and Enterprise LLMs







 YouTube

[\[RSS\]](#)

An [airhacks.fm](#) conversation with Antonio Goncalves ([@agoncal](#)) about:

#359 From SIMD to CUDA with TornadoVM







 YouTube

[\[RSS\]](#)

An [airhack](#) #356 AI/LLM Driven Development







 YouTube

[\[RSS\]](#)

An [airhacks.fm](#) conversation with Jonathan Ellis ([@spyced](#)) about

[brokk](#) as a Norse dwarf who forged Thor's hammer, [Java](#) Swing

journey from
Microsoft for
modern AI
assistance,
abstraction
standards and
[Entity](#) (BCE
integration
multi-tenant
potentially

GPU acceleration
CPU-bound
Alfonso's
[Vector API](#)
hybrid architecture
multiplication
compute
consumption

airhacks.TV

with the time machine, “100 episodes ago segment”

...any questions left?

airhacks.live

online, live virtual workshops

Continuous coding, interacting and sharing with [Adam Bien](#)

Live, Virtual Online Workshops, Winter 2026:

Building Local and Serverless LLM Agents, 15 December 2026

...the same agent, from laptop to cloud

LLM-Assisted Legacy System Migration to Modern Java, 17 December 2026

...turn legacy Java into maintainable, zero-dependency code

Tickets are also available from: airhacks.eventbrite.com and meetup.com/airhacks

 Prefer in-person, at Munich Airport?

airhacks.university brings hands-on workshops face-to-face. Small group, full day, all of us in one room.

Next: Architect-Grade Java with LLMs, 10 December 2026

by [Adam Bien](#)

airhacks.university

Back to MUC Airport



Real-world workshops
for **Java** developers



— RETURNS TO MUC

Architect-Grade Java with LLMs

*One day of coding, debating and
shipping Java with Adam Bien*

DATE

December 10, 2026

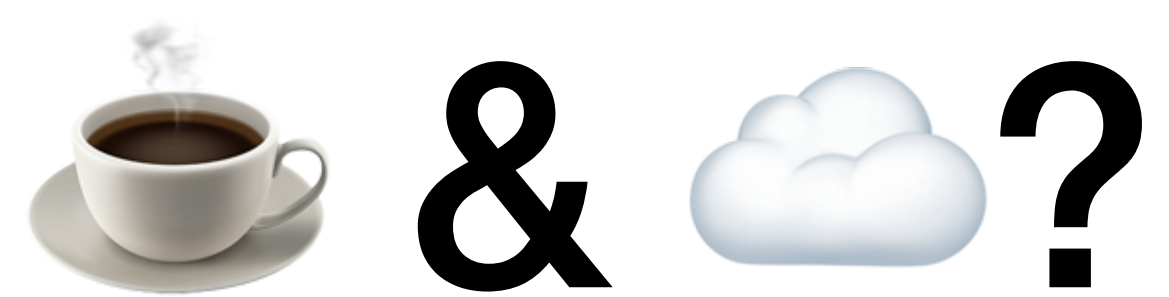
TIME

10:00 — 18:00 CET

LOCATION

Munich Airport

airhacks.university



Total					
	Energy		Time		Mb
(c) C	1.00	(c) C	1.00	(c) Pascal	1.00
(c) Rust	1.03	(c) Rust	1.04	(c) Go	1.05
(c) C++	1.34	(c) C++	1.56	(c) C	1.17
(c) Ada	1.70	(c) Ada	1.85	(c) Fortran	1.24
(v) Java	1.98	(v) Java	1.89	(c) C++	1.34
(c) Pascal	2.14	(c) Chapel	2.14	(c) Ada	1.47
(c) Chapel	2.18	(c) Go	2.83	(c) Rust	1.54
(v) Lisp	2.27	(c) Pascal	3.02	(v) Lisp	1.92
(c) Ocaml	2.40	(c) Ocaml	3.09	(c) Haskell	2.45
(c) Fortran	2.52	(v) C#	3.14	(i) PHP	2.57
(c) Swift	2.79	(v) Lisp	3.40	(c) Swift	2.71
(c) Haskell	3.10	(c) Haskell	3.55	(i) Python	2.80
(v) C#	3.14	(c) Swift	4.20	(c) Ocaml	2.82
(c) Go	3.23	(c) Fortran	4.20	(v) C#	2.85
(i) Dart	3.83	(v) F#	6.30	(i) Hack	3.34
(v) F#	4.13	(i) JavaScript	6.52	(v) Racket	3.52
(i) JavaScript	4.45	(i) Dart	6.67	(i) Ruby	3.97
(v) Racket	7.91	(v) Racket	11.27	(c) Chapel	4.00
(i) TypeScript	21.50	(i) Hack	26.99	(v) F#	4.25
(i) Hack	24.02	(i) PHP	27.64	(i) JavaScript	4.59
(i) PHP	29.30	(v) Erlang	36.71	(i) TypeScript	4.69
(v) Erlang	42.23	(i) Jruby	43.44	(v) Java	6.01
(i) Lua	45.98	(i) TypeScript	46.20	(i) Perl	6.62
(i) Jruby	46.54	(i) Ruby	59.34	(i) Lua	6.72
(i) Ruby	69.91	(i) Perl	65.79	(v) Erlang	7.20
(i) Python	75.88	(i) Python	71.90	(i) Dart	8.64
(i) Perl	79.58	(i) Lua	82.91	(i) Jruby	19.84

GraalVM™
reduces RAM footprint

<https://sites.google.com/view/energy-efficiency-languages/results?authuser=0>

LLMs and Java

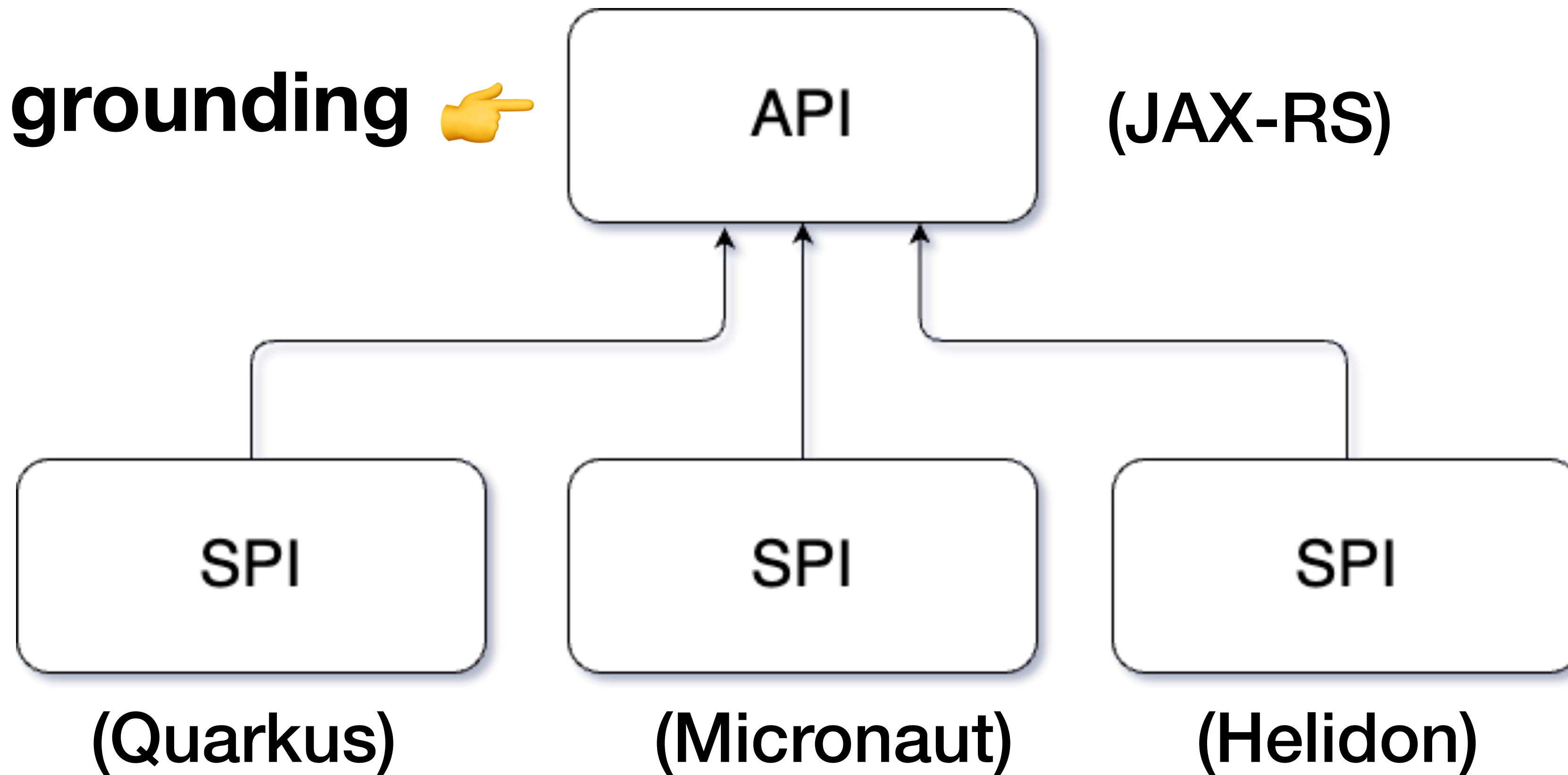
	Average	Python	Cpp	Java	JS	Go	Shell	Csharp	Dart	Elixir	Julia	Kotlin	Perl	PHP	Racket	R	Ruby	Rust	Scala	Swift	TS
Count		196	186	188	184	191	188	199	200	198	200	200	200	199	196	198	200	199	199	200	199
Current Upper Bound	74.8	63.3	74.7	78.7	59.2	69.1	70.7	88.4	78.0	97.5	78.0	89.5	64.5	52.8	88.3	74.2	79.5	61.3	77.4	78.0	61.3
Reasoning Mode																					
Claude Opus 4 (20250514)	52.4	40.3	44.1	55.9	38.6	37.2	51.6	74.9	54.0	80.3	55.5	72.5	44.5	28.1	68.9	52.5	61.0	38.7	50.3	50.0	47.2
Claude Sonnet 4 (20250514)	51.1	37.2	46.8	52.7	34.8	41.9	48.9	72.4	53.5	81.8	49.0	71.5	45.0	34.7	68.9	50.5	54.5	36.2	48.2	48.0	44.2
o3-high (20250416)	51.1	40.8	47.3	53.2	40.8	22.0	49.5	68.3	55.0	80.8	54.5	72.0	44.0	32.7	53.1	47.5	59.0	42.2	51.3	59.0	47.2
o4-mini (2025-04-16)	50.0	42.3	46.8	51.6	40.2	31.4	45.2	68.3	54.0	82.3	49.0	74.0	44.0	30.2	45.4	43.4	59.0	40.2	50.3	54.0	45.7
Grok-4	50.9	41.2	48.7	50.0	37.5	41.4	47.3	72.4	49.5	76.8	55.0	70.0	44.0	27.1	63.8	48.5	61.5	37.7	52.8	51.5	40.7
Gemini2.5 Pro	48.7	40.3	47.5	53.2	37.0	37.2	45.2	70.9	54.0	68.7	54.0	72.0	41.0	29.7	52.6	49.5	56.5	24.6	46.7	49.5	41.7
DeepSeek-R1-0528	50.2	38.8	43.6	52.7	35.9	38.7	46.8	75.4	52.5	77.3	52.0	70.0	39.0	28.6	56.1	50.5	58.5	37.2	51.8	55.0	41.2
Seed1.6-enabled (250615)	45.3	39.8	44.6	46.3	28.3	40.8	44.1	60.3	39.5	69.7	51.0	58.0	41.5	25.6	52.6	51.0	52.0	28.6	41.7	47.5	41.2
Seed1.6-Thinking-250715	45.0	40.3	45.2	50.0	33.2	38.2	39.9	67.3	36.5	67.7	51.0	61.0	41.0	26.1	51.0	44.9	55.5	27.6	37.2	46.5	38.7
Seed1.6-Thinking-250615	44.7	38.8	47.0	49.5	38.0	31.4	38.8	62.3	41.0	70.7	45.0	68.0	39.0	25.1	47.5	47.5	50.5	30.7	39.7	41.5	40.2
GLM-4.5-enable	46.6	41.0	43.2	47.9	34.8	37.8	43.9	70.5	42.0	72.5	47.5	66.0	43.5	28.6	50.0	45.0	54.5	31.6	41.0	46.0	42.2
GLM-4.5-Air-enable	40.8	39.3	37.6	39.4	31.0	39.8	36.7	66.3	38.0	61.5	42.0	53.0	40.5	27.1	40.3	39.0	47.0	25.1	30.5	38.5	42.7
ERNIE-X1-Turbo-32K	39.6	39.4	17.8	33.2	32.6	37.4	33.9	46.0	33.0	68.9	54.0	49.5	39.5	23.9	45.3	44.3	48.0	20.8	40.4	44.0	37.7
Qwen3-235B-A22B-Thinking-2507	47.7	37.8	41.9	48.4	39.7	39.8	45.2	71.9	46.0	79.8	48.5	58.0	40.5	29.1	56.6	49.0	55.0	35.7	40.4	46.0	44.2
Qwen3-235B-A22B	45.9	36.7	43.5	47.3	36.4	37.7	42.0	70.9	45.5	68.7	46.0	60.0	39.0	29.1	52.0	47.0	56.5	31.7	43.7	41.5	41.7
Qwen3-32B	41.7	37.8	38.7	39.9	32.6	36.1	39.4	67.8	34.5	65.2	42.5	52.0	40.5	27.6	37.8	44.9	47.0	28.1	37.2	42.0	40.2
Qwen3-14B	37.6	37.8	35.5	35.1	30.4	30.4	36.2	60.8	29.0	62.1	34.5	44.5	37.5	23.1	44.9	36.9	43.5	24.6	28.6	36.0	38.7
Qwen3-8B	28.5	28.1	22.6	21.8	28.3	29.3	27.1	52.8	21.0	43.9	29.0	36.0	35.5	18.6	13.3	30.8	37.0	12.6	21.1	22.0	37.7
Qwen3-4B	24.3	27.6	17.7	22.3	25.5	24.1	28.2	42.2	13.0	33.3	20.0	29.5	34.5	16.1	11.7	23.7	27.5	8.5	20.1	20.0	39.2
Qwen3-1.7B	11.2	16.8	5.4	4.8	12.5	9.9	11.7	19.6	7.0	20.7	11.0	9.0	19.5	7.5	5.6	9.6	21.0	0.0	2.5	10.0	19.6
Non-Reasoning Mode																					
Claude Opus 4 (20250514)	50.9	37.8	45.7	50.0	38.0	35.6	47.3	73.9	57.0	82.3	55.0	75.5	43.0	26.6	64.8	47.0	54.0	38.2	46.7	51.5	46.7
Claude Sonnet 4 (20250514)	49.3	35.7	47.3	52.7	38.0	37.7	47.9	72.9	51.0	74.2	51.0	72.0	44.0	30.7	63.8	44.4	51.5	35.2	45.2	45.5	44.2
GPT4.1 (2025-04-14)	48.0	37.2	46.8	48.9	34.8	37.2	36.7	74.4	46.5	76.8	50.0	72.0	43.5	29.2	50.5	42.4	54.0	37.2	44.2	49.5	46.2
GPT4o (2024-11-20)	41.1	33.7	37.1	43.2	34.8	30.9	29.3	65.3	43.5	62.6	36.0	67.0	43.0	26.6	37.2	32.8	43.0	29.6	38.2	45.0	39.2
Gemini2.5 Flash	45.7	39.3	44.1	50.0	33.2	33.0	37.8	68.3	49.5	64.0	47.5	70.0	39.5	24.1	38.3	51.5	53.0	36.2	44.2	46.5	41.2
DeepSeek-V3-0324	48.1	36.7	48.4	52.7	31.5	34.6	37.8	72.9	48.0	75.8	49.0	69.0	42.5	28.1	59.2	45.0	52.5	37.2	46.7	48.0	43.7
DeepSeek-Coder-V2	37.7	29.1	34.9	34.0	27.7	29.8	31.4	63.8	33.5	60.6	37.5	58.5	35.5	25.1	41.8	35.4	45.0	22.6	33.2	38.0	35.7
DeepSeek-Coder-33B-Instruct	28.5	25.0	24.2	29.3	24.5	29.8	22.3	54.8	17.5	67.7	14.5	52.0	29.5	19.1	28.1	18.7	33.0	8.0	24.1	18.0	29.1
DeepSeek-Coder-6.7B-Instruct	20.5	18.9	12.9	19.7	19.6	21.5	16.0	44.2	11.5	47.5	15.5	45.5	21.5	10.6	15.3	13.1	27.5	6.0	11.1	8.0	23.6
Kimi-K2-0711-Preview	47.8	38.8	42.5	47.9	37.5	31.4	40.4	75.9	50.0	80.3	52.0	68.0	41.5	28.1	57.1	40.4	52.5	36.7	42.7	47.0	42.2
Hunyuan-TurboS-20250716	43.8	34.2	34.9	47.9	32.6	34.6	38.3	64.8	44.5	70.7	47.0	62.0	42.0	30.2	45.9	39.9	53.0	30.7	39.2	39.5	42.2
Hunyuan-Coder-7B-Preview	33.4	25.5	26.9	34.0	27.2	21.5	29.3	54.8	36.0	60.1	30.5	57.0	34.0	22.2	37.8	27.3	40.5	11.1	31.2	27.6	31.2
Seed1.6-disabled (250615)	42.9	35.2	40.3	46.8	32.6	34.6	35.1	70.9	42.5	69.7	45.0	62.0	39.5	23.1	49.5	40.4	46.5	28.1	32.7	40.0	42.7
Seed-Coder-8B-Instruct	32.3	23.5	23.7	33.5	28.8	22.5	20.7	54.8	30.5	57.1	33.0	52.5	34.0	25.1	36.7	28.3	35.5	15.6	29.6	28.0	31.2
Qwen3-Coder-480B-A35B-Instruct	44.8	39.4	41.1	51.1	27.9	31.4	41.1	63.0	36.5	73.7	49.5	63.1	41.0	27.2	56.3	42.7	51.5	25.4	42.1	47.5	41.9
Qwen3-235B-A22B-Instruct-2507	43.1	35.7	38.2	49.5	29.3	33.5	40.4	67.3	39.5	59.1	46.0	59.5	44.5	26.1	49.5	44.0	46.5	24.6	37.7	46.0	43.2
Qwen3-32B	31.0	26.5	21.5	29.5	28.0	25.5	24.0	59.3	27.5	52.0	28.0	45.0	34.5	21.6	28.6	22.7	36.5	16.1	26.6	30.0	35.2
Qwen3-14B	28.6	24.5	22.6	32.4	27.2	16.8	23.9	50.8	21.0	42.4	22.5	42.0	34.5	24.6	28.1	26.3	33.5	17.1	20.6	26.5	32.7
Qwen3-8B	23.3	22.4	11.3	25.0	22.8	18.3	22.3	42.2	17.0	41.4	18.5	36.0	29.5	18.1	23.5	16.2	27.0	7.5	19.1	17.5	29.1
Qwen3-1.7B	7.9	8.7	1.1	2.7	8.2	3.7	11.7	9.5	3.5	17.2	6.0	11.5	15.5	7.5	4.6	7.6	14.5	0.5	3.0	6.0	14.6
Qwen2.5-Coder-32B	35.8	29.6	27.4	33.0	29.9	23.0	29.3	58.3	34.5	59.6	35.5	56.0	38.5	26.1	35.7	31.3	40.0	23.1	29.6	39.0	35.2
Qwen2.5-72B	34.3	33.2	24.7	31.9	28.8	19.9	31.9	54.3	34.0	65.7	30.5	50.5	30.0	22.6	39.3	27.3	42.0	23.1	32.7	29.5	33.2
Qwen2.5-Coder-7B	22.5	19.9	8.6	22.3	21.2	12.6	21.3	38.7	18.5	47.0	18.0	39.0	27.5	15.1	24.0	17.7	29.5	7.0	19.1	17.0	24.6
Qwen2.5-Coder-1.5B	10.3	12.2	2.7	4.8	12.0	7.3	14.4	17.6	6.5	35.4	4.0	16.5	15.0	5.0	7.1	7.6	15.5	1.0	4.5	5.0	11.1
OpenCoder-8B	19.3	14.3	9.7	19.1	12.5	17.3	21.3	33.2	15.5	34.3	15.5	30.0	26.5	14.6	15.8	17.2	29.5	6.0	17.1	15.0	21.1

Table 4: Pass@1 (%) performance of different models for AutoCodeBench. *Current Upper Bound* represents the Pass@1 value calculated by taking the union of problems correctly solved by all models.

<https://github.com/Tencent-Hunyuan/AutoCodeBenchmark/tree/main>

"No Variations"-The Trick

LLM grounding 📌



...and WebSphere, Payara, JBoss, WebLogic

Normative Language

RFC 2119

<https://www.ietf.org/rfc/rfc2119.txt>

Key words for use in RFCs to Indicate Requirement Levels

Status of this Memo

This document specifies an Internet Best Current Practices for the Internet Community, and requests discussion and suggestions for improvements. Distribution of this memo is unlimited.

Abstract

In many standards track documents several words are used to signify the requirements in the specification. These words are often capitalized. This document defines these words as they should be interpreted in IETF documents. Authors who follow these guidelines should incorporate this phrase near the beginning of their document:

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119.

Note that the force of these words is modified by the requirement level of the document in which they are used.

1. **MUST** This word, or the terms "REQUIRED" or "SHALL", mean that the definition is an absolute requirement of the specification.
2. **MUST NOT** This phrase, or the phrase "SHALL NOT", mean that the definition is an absolute prohibition of the specification.
3. **SHOULD** This word, or the adjective "RECOMMENDED", mean that there may exist valid reasons in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed before choosing a different course.
4. **SHOULD NOT** This phrase, or the phrase "NOT RECOMMENDED" mean that there may exist valid reasons in particular circumstances when the particular behavior is acceptable or even useful, but the full implications should be understood and the case carefully weighed before implementing any behavior described with this label.

- Copyright
 - Eclipse Foundation Specification License
 - Disclaimers
- 1. Introduction
 - 1.1. Status
 - 1.1.1. Support for @Context Injection
 - 1.2. Goals
 - 1.3. Non-Goals
 - 1.4. Conventions
 - 1.5. Terminology
 - 1.6. Project Team Members
 - 1.7. Acknowledgements
- 2. Applications
 - 2.1. Configuration
 - 2.2. Verification
 - 2.3. Publication
 - 2.3.1. Java SE
 - 2.3.2. Servlet
 - 2.3.3. Other Container
- 3. Resources
 - 3.1. Resource Classes
 - 3.1.1. Lifecycle and Environment
 - 3.1.2. Constructors
 - 3.2. Fields and Bean Properties
 - 3.3. Resource Methods
 - 3.3.1. Visibility
 - 3.3.2. Parameters
 - 3.3.3. Return Type
 - 3.3.4. Exceptions
 - 3.3.5. HEAD and OPTIONS

1.4. Conventions

The keywords `MUST`, `MUST NOT`, `REQUIRED`, `SHALL`, `SHALL NOT`, `SHOULD`, `SHOULD NOT`, `RECOMMENDED`, `MAY`, and `OPTIONAL` in this document are to be interpreted as described in RFC 2119[10].

Java code and sample data fragments are formatted as shown in figure [1.1]:

```
package com.example.hello;

public class Hello {
    public static void main(String args[]) {
        System.out.println("Hello World");
    }
}
```

URIs of the general form `http://example.org/...` and `http://example.com/...` represent application or context-dependent URIs.

All parts of this specification are normative, with the exception of examples, notes and sections explicitly marked as Normative'. Non-normative notes are formatted as shown below.

Note: *This is a note.*

1.5. Terminology

Resource class

A Java class that uses JAX-RS annotations to implement a corresponding Web resource, see Chapter [resources].

Root resource class

A *resource class* annotated with `@Path`. Root resource classes provide the roots of the resource class tree and provide access to sub-resources, see Chapter [resources].

<https://jakarta.ee/specifications/restful-ws/4.0/>

- 1.3. Non-Goals
- 1.4. Conventions
- 1.5. Terminology
- 1.6. Project Team Members
- 1.7. Acknowledgements
- 2. Applications
 - 2.1. Configuration
 - 2.2. Verification
 - 2.3. Publication
 - 2.3.1. Java SE
 - 2.3.2. Servlet
 - 2.3.3. Other Container
- 3. Resources
 - 3.1. Resource Classes
 - 3.1.1. Lifecycle and Environment
 - 3.1.2. Constructors
 - 3.2. Fields and Bean Properties
 - 3.3. Resource Methods
 - 3.3.1. Visibility

The value of a URI path parameter is available for injection via `@PathParam` on a field, property or method parameter. Note that if a URI template is used on a method, a path parameter injected in a field or property may not be available (set to `null`). The following example illustrates this scenario:

```
@Path("widgets")
public class WidgetsResource {
    @PathParam("id") String id;

    @GET
    public WidgetList getWidgets() {
        ... // id is null here
    }

    @GET
    @Path("{id}")
    public Widget findWidget() {
        return new WidgetResource(id);
    }
}
```

API

<https://jakarta.ee/specifications/restful-ws/4.0/>



Contents

[What you'll learn](#)[Getting started](#)[Try what you'll build](#)

Creating a RESTful application

[Creating the RESTful resource](#)[Configuring Liberty](#)[Running the application](#)[Testing the service](#)[Running the tests](#)[Guide Attribution](#)[Great work! You're done!](#)

Tags

[jakarta-ee](#) [microprofile](#)[run-in-cloud](#)

SPI

Creating a RESTful application

Navigate to the `start` directory to begin.

When you run Open Liberty in **dev mode**, dev mode listens for file changes and automatically recompiles and deploys your updates whenever you save a new change. Run the following goal to start Open Liberty in dev mode:

WINDOWS **MAC** **LINUX**

```
./mvnw liberty:dev
```

After you see the following message, your Liberty instance is ready in dev mode:

```
*****  
*   Liberty is running in dev mode.
```

Dev mode holds your command-line session to listen for file changes. Open another command-line session to continue, or open the project in your editor.

Jakarta Restful Web Services defines two key concepts for creating REST APIs.

[SystemApplication.java](#)[PropertiesResource.java](#)[server.xml](#)[pom.xml](#)

```
package io.openliberty.guides.rest;  
  
import java.util.Properties;  
  
import jakarta.ws.rs.GET;  
import jakarta.ws.rs.Path;  
import jakarta.ws.rs.Produces;  
import jakarta.ws.rs.core.MediaType;  
  
@Path("properties")  
public class PropertiesResource {  
  
    @GET  
    @Produces(MediaType.APPLICATION_JSON)  
    public Properties getProperties() {  
        return System.getProperties();  
    }  
}
```

API



Helidon
Version: (4.4.0)

Helidon Documentation

Get Started

Helidon MP

Introduction

Guides

AI

Bean Validation

Configuration SPI

Data Repository

Fault Tolerance

GraphQL Server

gRPC

Health Checks

Integrations

JAX-RS/Jersey

Configuring the Application

Jakarta REST

Jakarta REST Client

Helidon Connector

≡

🔍

MP — Jakarta REST

}

The served resources can be reached through `/my-application/{myResources}` endpoint. It can be overridden by configuration file.

Example of custom application path using `.yaml` file:

```
io.helidon.examples.MyApplication:
  routing-path:
    path: "/my-application"
```

The same configuration works for `.properties` file:

```
io.helidon.examples.MyApplication.routing-path.path=/my-application
```

If an `Application` is not provided, a *synthetic* subclass is created and can be configured using this property:

```
jakarta.ws.rs.core.Application.routing-path.path=/my-application
```

Access to Application Instances

Jakarta REST provides access to the `Application` subclass instance via injection using `@Context`. This form of access is still supported in Helidon but this is not enough if two or more subclasses are present. Given that support for two or more `Application` subclasses is a Helidon extension, a new mechanism is provided via the `ServerRequest` 's context object as shown next.

```
@Path("myresource")
public class MyResource {

    @GET
    public void get(@Context ServerRequest serverRequest) {
        Application app = serverRequest.context().get(Application.class).get();
    }
}
```

API

https://helidon.io/docs/v4/mp/jaxrs/jaxrs-applications

What is Quarkus REST?

Quarkus REST is a new [Jakarta REST \(formerly known as JAX-RS\)](#) implementation written from the ground up to work on our common [Vert.x](#) layer and is thus fully reactive, while also being very tightly integrated with Quarkus and consequently moving a lot of work to build time.

You should be able to use it in place of any Jakarta REST implementation, but on top of that it has great performance for both blocking and non-blocking endpoints, and a lot of new features on top of what Jakarta REST provides.

Writing endpoints

Getting started

Add the following import to your build file:

pom.xmlbuild.gradle

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-rest</artifactId>
</dependency>
```

You can now write your first endpoint in the `org.acme.rest.Endpoint` class:

```
package org.acme.rest;

import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;

@Path("")
public class Endpoint {

    @GET
    public String hello() {
        return "Hello, World!";
    }
}
```

API

What is Quarkus REST?

Writing endpoints

Getting started

Terminology

Declaring endpoints: URI mapping

Declaring endpoints: HTTP methods

Declaring endpoints: representation / content types

Accessing request parameters

Declaring URI parameters

Accessing the request body

Handling Multipart Form data

Returning a response body

Setting other response properties

Redirect support

Async/reactive support

Streaming support

Concurrent stream element processing

Returning multiple JSON objects

Server-Sent Event (SSE) support

Controlling HTTP Caching features

Accessing context objects

JSON serialisation

XML serialisation

Web Links support

CORS filter

More advanced usage

Execution model, blocking, non-blocking

Exception mapping

Request or response filters

Readers and Writers: mapping entities and HTTP bodies

Reader and Writer interceptors

SPI

LLM training

LLM training

- LLM providers (also) use free data on the internet to train the LLMs
- High quality, structured data is preferred
- Common Crawl (commoncrawl.org) contains publicly crawlable data
- jakarta.ee and microprofile.io are included

Common Crawl maintains a free, open repository of web crawl data that can be used by anyone.

Common Crawl is a 501(c)(3) non-profit founded in 2007.

We make wholesale extraction, transformation and analysis
of open web data accessible to researchers.

Overview

Over 300 billion pages spanning 19 years.

Free and open corpus since 2007.

Cited in over 10,000 research papers.

3–5 billion new pages added each month.

reducing hallucinations

reducing hallucinations

- tech stack in pre-trained data
- "don't fight the weights"
- small context
- grounded to stable APIs
- small API surface

reducing hallucinations

- compilation gate
- UT gate
- IT gate
- ST gate
- short SKILLS.md and AGENTS.md
- test: short prompt 🙏 good code

Claude's Opinion

```
> /clear
```

```
> How well do you know Java?
```

🗨️ Pretty well – it's core to a lot of what I do, and especially well-suited to this project's world.

Concretely:

- **The language, current:** through Java 25 – records, sealed types, pattern matching (incl. for `switch` and record deconstruction), virtual threads, structured concurrency, `switch/instanceof` patterns, text blocks, sequenced collections, the module system, etc. I lean toward modern idioms rather than legacy boilerplate.
- **The platform:** JVM memory model, GC behavior, concurrency (`java.util.concurrent`, `CompletableFuture`, virtual threads vs. platform threads), streams, NIO, the `HttpClient`.
- **This stack specifically:** MicroProfile (Config, Health, Metrics, Fault Tolerance, OpenAPI, REST Client), Jakarta EE (CDI, JAX-RS, JSON-B/JSON-P, Bean Validation, JPA), Quarkus, and the BCE (Boundary-Control-Entity) architecture your project is organized around.
- **Tooling & practice:** Maven, JUnit, testing layers (unit / integration / system), dependency hygiene.

Where I'm weaker / will verify rather than guess: exact behavior of very new preview features, precise version-to-version API availability, and anything where a library's current API matters – I'll check the actual code or docs instead of trusting memory.

› Does choosing the technology Java, MicroProfile and Jakarta EE, as well as the BCE architecture, help to save tokens and inference costs?

📱 Yes – measurably, with this session as evidence:

Concrete token wins observed this session:

- **Zero failed builds and zero failed test runs across eight feature turns.** Every `mvn test` / `mvn verify` returned `BUILD SUCCESS` on the first attempt. The typical cost of a retry – re-reading failing test output, locating the broken line, editing, re-running – is 5–10k tokens per round. At one retry-per-feature avoided × eight features, that's a ~40–80k-token differential against a normal trial-and-error flow.
- **Almost no search.** I read `AGENTS.md` and the `health BC` once at the start and never grepped for anything afterward. BCE predicts every file path; MicroProfile predicts every annotation. On an unstructured project the same eight turns would have spent 30–50k tokens just locating the right files.
- **No documentation lookup mid-task.** JSON-P, JAX-RS, CDI, MP REST Client are stable, training-data-dense APIs – `Json.createObjectBuilder()`, `Response.accepted()`, `@RegisterRestClient(configKey=...)` are muscle memory. I never needed to `WebFetch` a spec. Compare with hitting a fresh extension or a niche newer framework where the doc read is 10–20k tokens per call.
- **Compile-time guarantees collapse round-trips.** Static typing catches contract errors before any test runs. With dynamic typing I'd run the test, parse a runtime `TypeError`, re-read, patch – three turns where Java needs zero.

why code quality matters

today's code is tomorrow's context

Generated code becomes input for the next generation

Poor quality compounds as the codebase grows

Structure keeps relevant context small enough to fit

Locality principle (BCs) limits blast radius

Local contracts and tests give a verifiable oracle

today's code is tomorrow's context

Quality drives speed: fewer tokens, fewer retries

Testability drives reliability: containment plus verification

The unit of change is a feature, not a layer

Degradation shows at scale, not in small demos

Intent Engineering

Functional Intent Engineering

is domain-specific

describes what the system should do

maps to features and behavior

is verifiable through tests

translates directly to code

missing functional intent produces incomplete features

functional correctness is binary: works or fails

Non-functional Intent Engineering

intent describes how well it should do it

intent maps to quality attributes

is verifiable through metrics

intent translates to architecture decisions

scalability, security, observability, resilience

correctness is a spectrum

Non-functional Intent Engineering

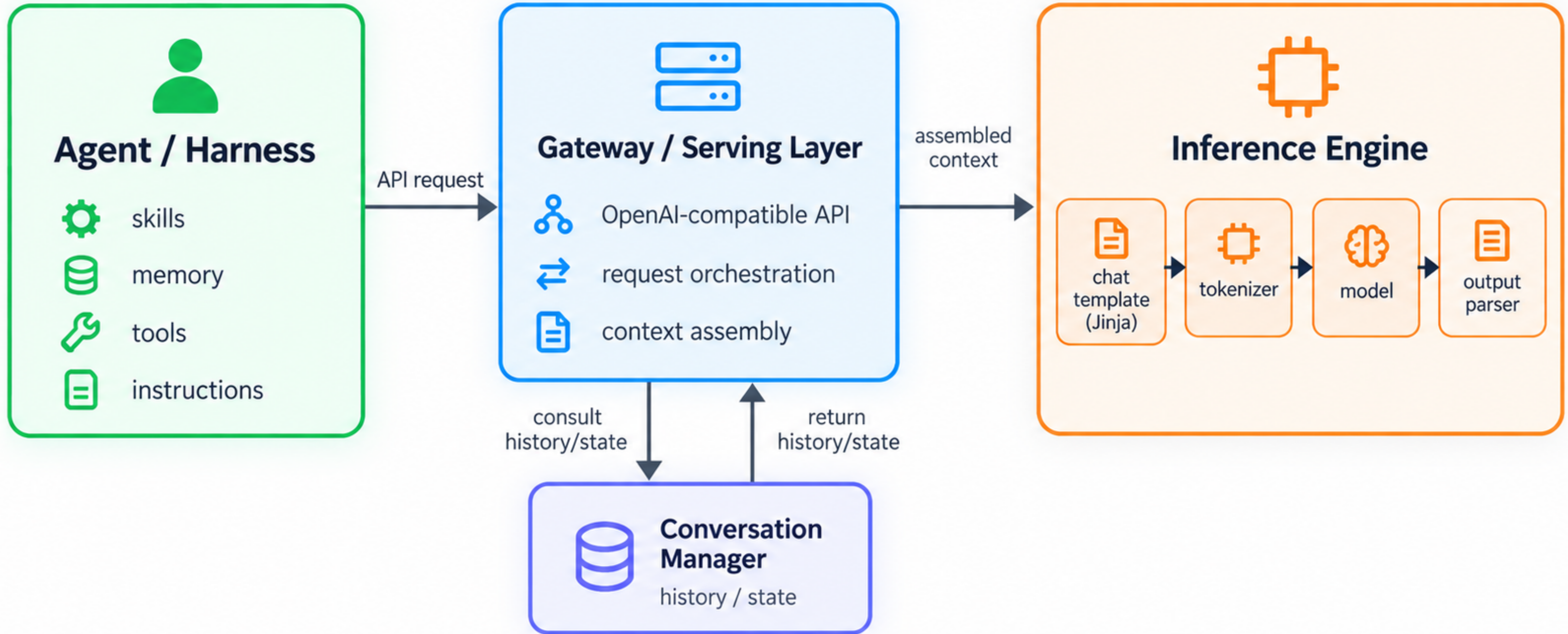
intent is verifiable through metrics

missing non-functional intent produces fragile systems

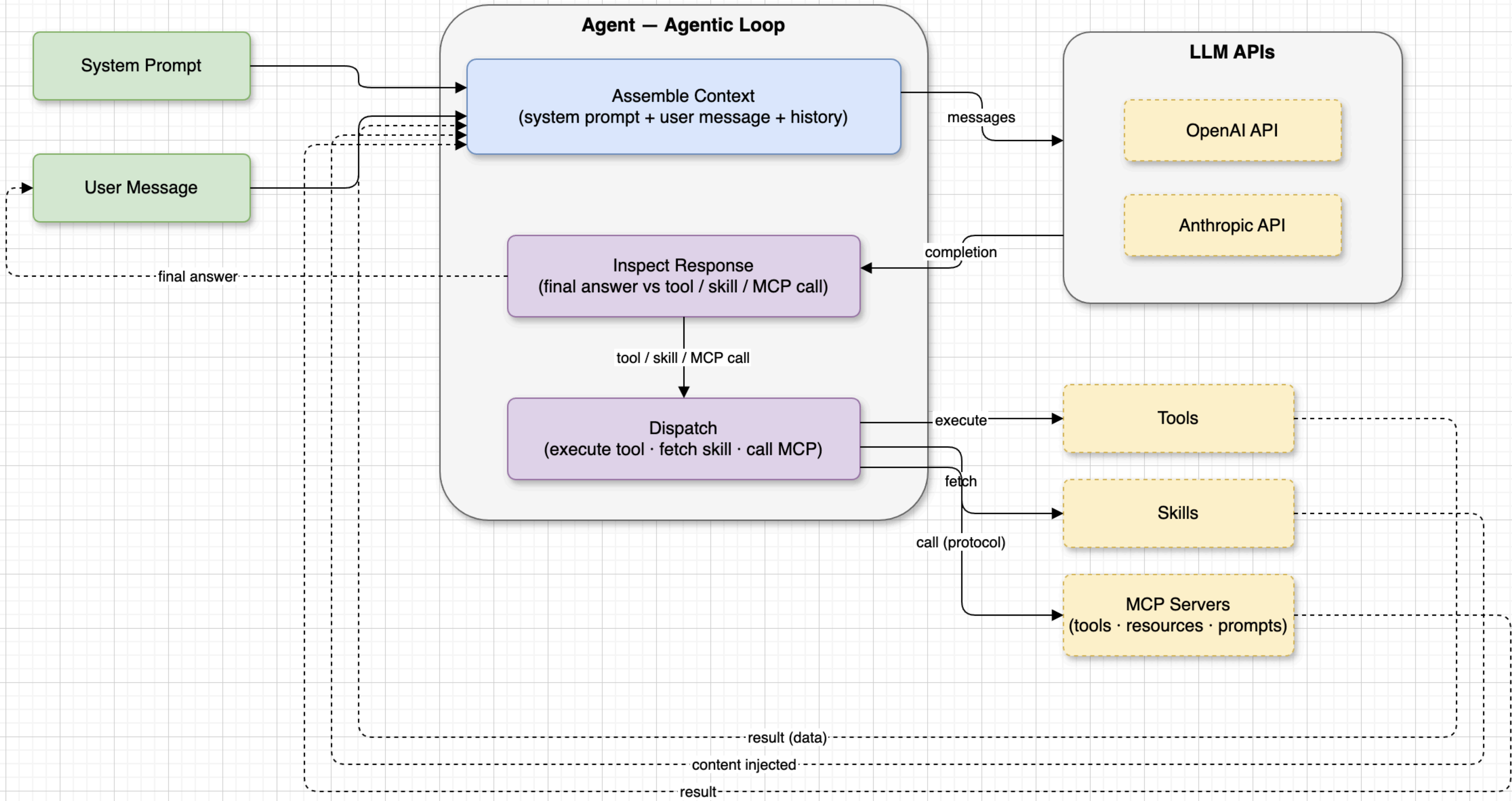
non-functional intent often ignored without explicit prompting

the agentic loop

Three Layers of LLM Execution



Two compiler stages: harness compiles concepts into API requests; the inference engine compiles requests into model tokens.



LLM-first tech stack

LLM-first

Favor mainstream, long-lived languages and frameworks

Code to standard APIs, not implementations

Prefer the conventional option over novel or niche

Choose frameworks with one canonical way

Prefer convention over configuration

Use standard naming and directory layouts

LLM-first

Require mature tooling

Prefer static typing for deterministic error catching

Keep tooling CLI-drivable and headless for agents

Optimize for fast build and test loops

Minimize dependencies to reduce version-specific errors

Prefer standard library over third-party libraries

LLM-first

Avoid fast-churning libraries with drifting APIs

Apply named patterns: e.g. BCE, GoF

Avoid custom DSLs and custom abstractions

Use small, self-contained, composable units

Co-locate the contract with the code

Pin versions and prefer LTS releases

Prefer pre-cutoff features

Spend innovation budget on composition

LLM-first summary

Use model's knowledge: the model generates from its priors (it knows EARS, BCE, package-info, the stack)

Ground to standards: that output is shaped/anchored to those same conventions, so it's constrained, durable, and human-legible

Let deterministic tools judge: the standards are exactly what tools can check (EARS → tests, compiler, test loop)

Specification-Driven Development

SDD

Specification-driven development is a [software development](#) approach in which specifications are used to develop software.

Specification-driven development is a type of documentation-driven development, along with [model-driven development](#), [model transformation](#), and [round-trip engineering](#).^[1]

https://en.wikipedia.org/wiki/Specification-driven_development

SDD

"Spec-driven development is the discipline where an executable-in-intent specification is the source of truth, and code is derived from and continuously reconciled against it - rather than the spec being upfront documentation that rots the moment coding starts."

by Claude Opus 4.8

SDD

"Spec-Driven Development is a development style where the specification is the primary artifact, and code, tests, documentation, and implementation tasks are derived from it, validated against it, and updated only through explicit changes to it."

by GPT 5.5

SDD

"Spec-Driven Development is a contract-first approach where a capability specification (the "spec") defines the behavior, interfaces, and requirements of a feature or business component before any implementation code is written.

The spec is the source of truth and the boundary contract — implementation code converges to satisfy it, tests verify compliance against it, and it serves as living documentation that evolves with the system."

by mistral medium 3.5

BCE / ECB

Boundary Component Entity (BCE)

- BCE emerged from his work at Ericsson in the late 1980s and became part of the Objectory methodology
- BCE (also known as ECB—Entity-Control-Boundary) was introduced by Ivar Jacobson in 1992 in his book "Object-Oriented Software Engineering: A Use Case Driven Approach"
- Jacobson later became one of the "Three Amigos" (alongside Grady Booch and James Rumbaugh) who created UML

BCE

BCE organizes code into business components, each a vertical slice of one capability.

Boundary: the entry point; exposes the component to the outside world (UI, REST, CLI).

Control: the "how"; orchestration and business logic that isn't a single entity's job.

Entity: the domain data and its rules.

BCE

Business component: one capability, named for it, containing its own boundary/control/entity.

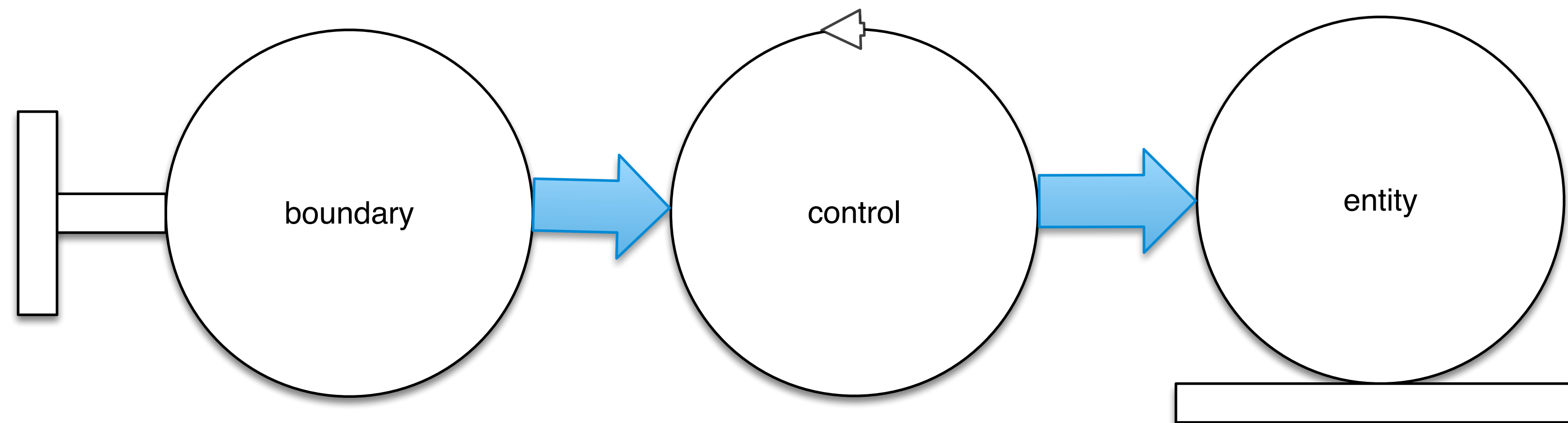
Dependency direction: Boundary -> Control -> Entity; never the reverse.

No technical layering: slice by capability (checkout, inventory), not by tier.

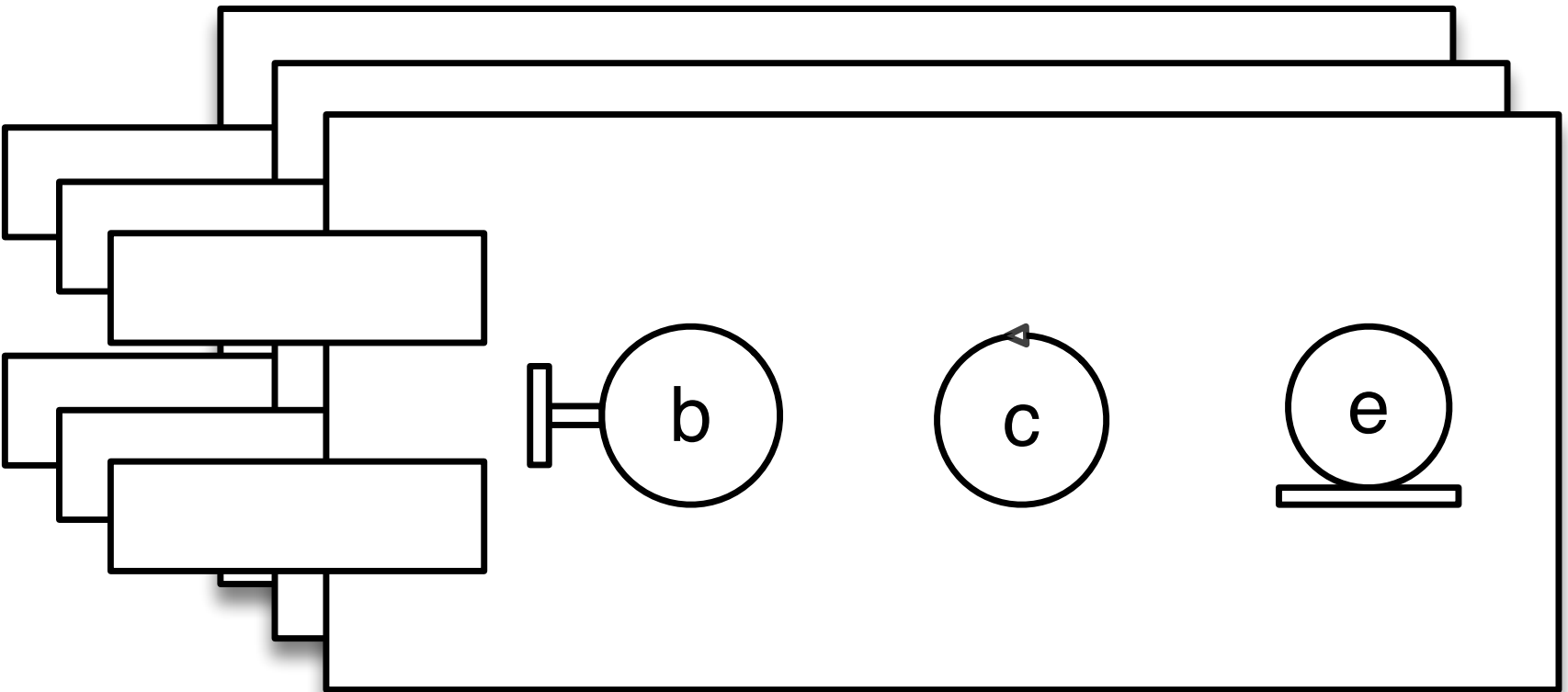
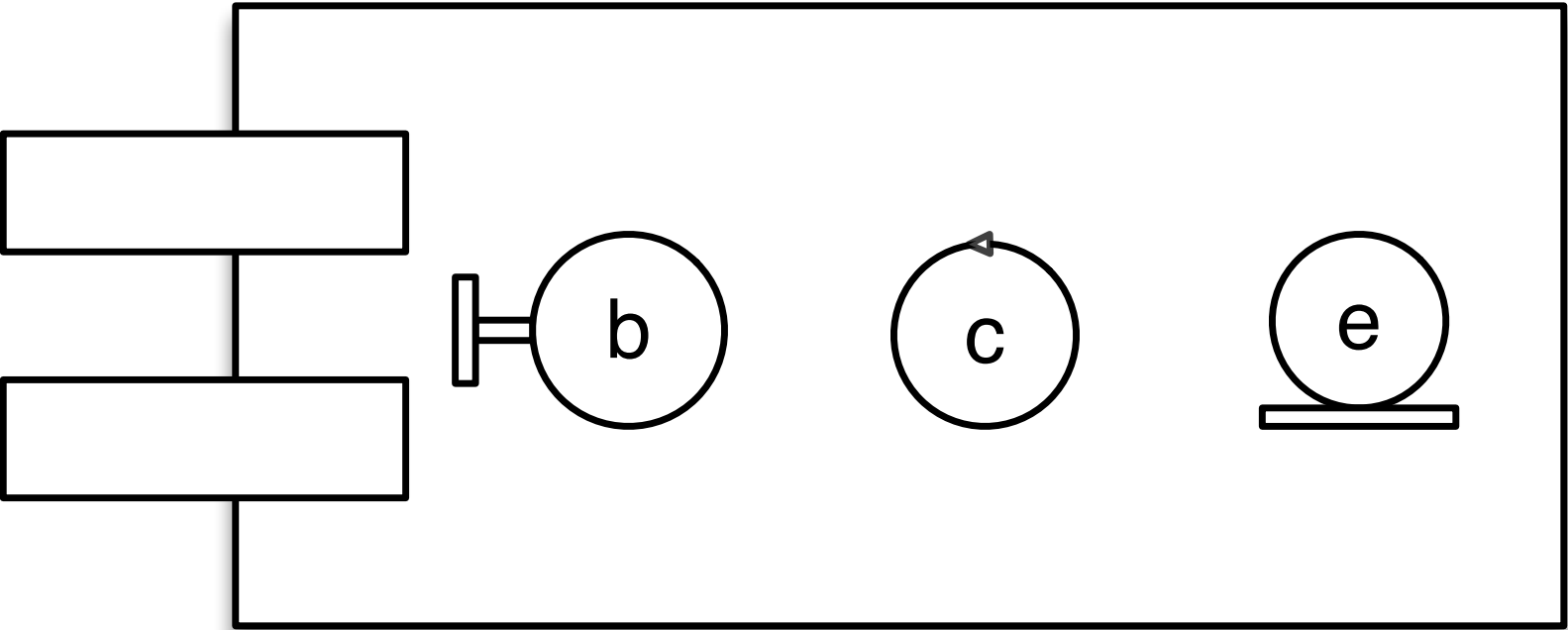
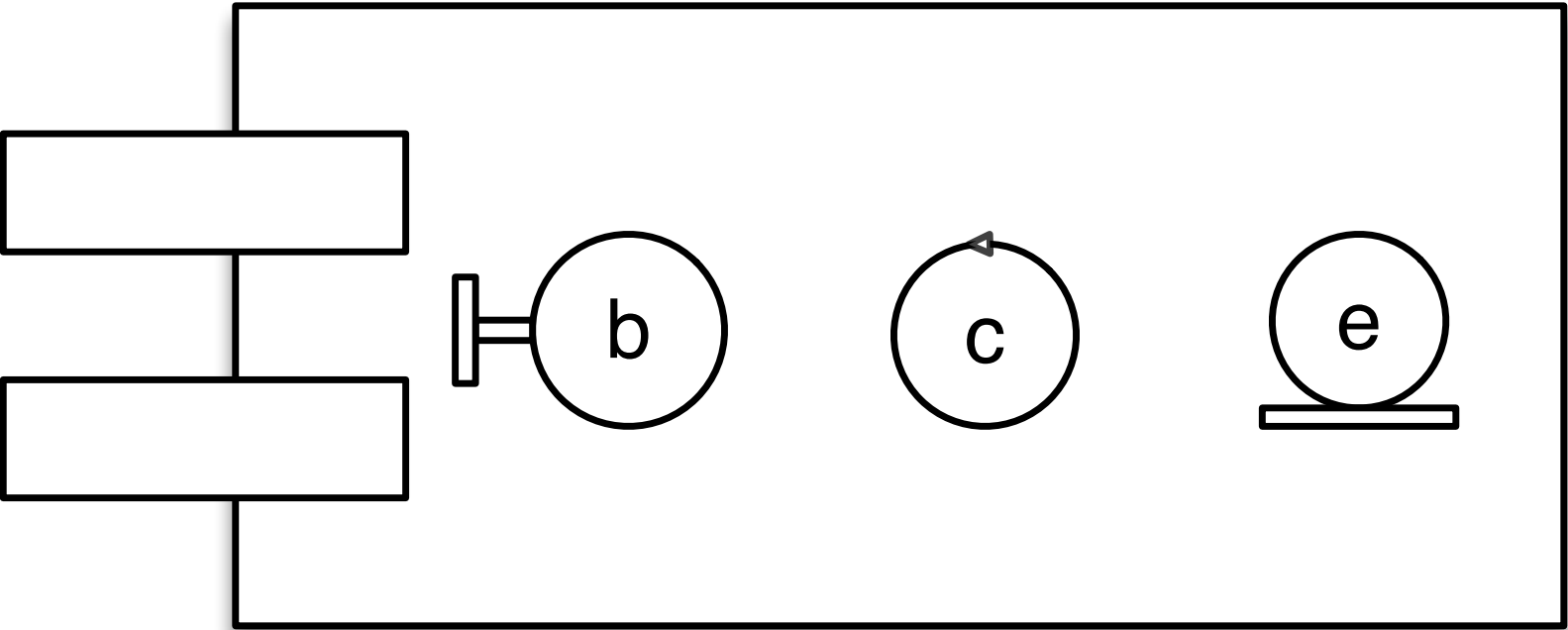
Naming: name for the domain; ban meaningless suffixes like *Impl / *Service.

The package doc is the documentation: package-info.java (/// Markdown, JEP 467) or package-info.md, co-located with the business component it describes.

BCE



Boundary Control Entity



BCE advantages

BCE advantages

BCE groups code by business capability, not technical layer

Each business component keeps boundary, control, entity together

One component loads as a single context slice

Changes stay inside one component, small blast radius

BCE advantages

One folder per capability makes code findable

The boundary is the contract and entry point

Each component carries its own tests, locally verifiable

Remove a capability by deleting its component

BCE advantages

Named responsibilities give clear ownership

Well-known pattern, familiar to the model

Puts things that change together, together

One component, two maintainers: human and model

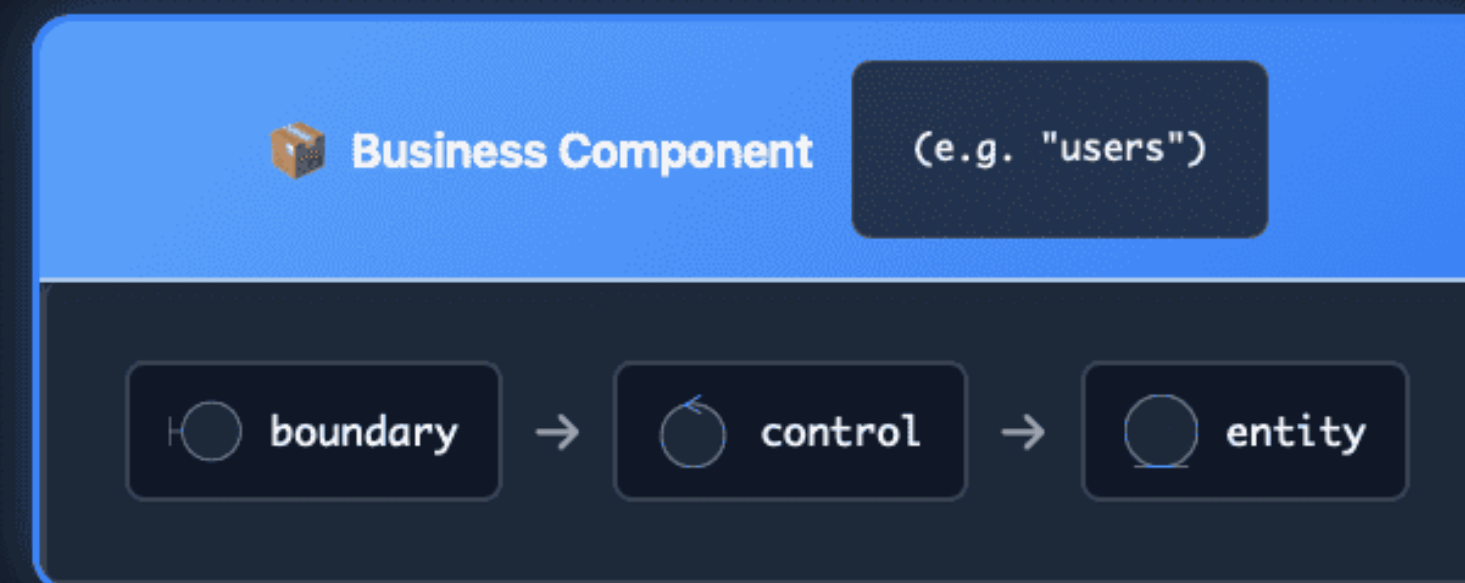
bce.design

Boundary Control Entity Architecture

Focus on building understandable applications with a strong emphasis on domain logic

Overview

The Boundary-Control-Entity (BCE/ECB) pattern is a software architecture pattern that organizes code into Business Components. A Business Component is a package or namespace comprising three distinct layers, each with specific responsibilities. Business components adhere to the principles of maximal cohesion and minimal coupling, and are named after their domain responsibilities.



<https://bce.design>

airails.dev

airails.dev

Guidelines for Agentic development.

"Put your AI on rails."

[BCE](#) architecture · Java · Web standards

skill.md

Download conventions and guidelines for AI-assisted development. Place in your AI coding agent's skills directory.

[java-conventions](#) ↓

[details](#)

Generic Java 25 code conventions, composable across all Java contexts

[sbce](#) ↓

NEW

[sbce.space](#)

[details](#)

Spec-driven BCE: one capability spec equals one business component

[microprofile-server](#) ↓

[details](#)

MicroProfile services with BCE architecture

[java-cli-app](#) ↓

[details](#)

Java 25 CLI application in source-file mode

[java-cli-script](#) ↓

[details](#)

Executable Java 25 script for system-wide use via PATH

[static-web](#) ↓

[details](#)

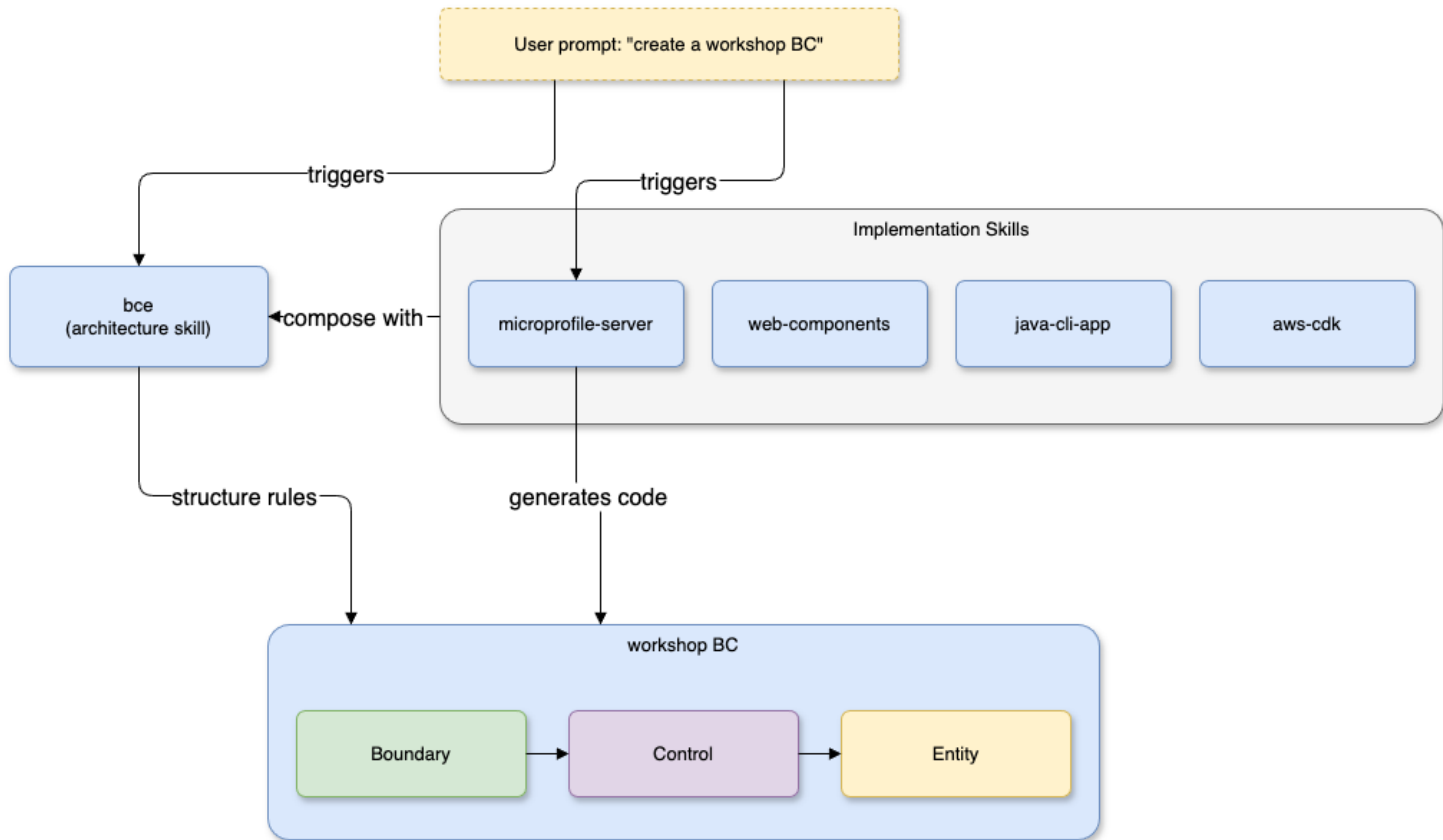
Static web development with semantic HTML and CSS

[web-components](#) ↓

[details](#)

SPAs with web components, lit-html, Redux Toolkit and BCE layering

<https://airails.dev>



EARS

Easy Approach to Requirements Syntax

Created by Alistair Mavin at Rolls-Royce

Reduces ambiguity, vagueness, and omission

Constrains requirements into fixed sentence templates

Every requirement uses the keyword "shall"

Easy Approach to Requirements Syntax

Ubiquitous: The system shall <response>

State-driven: While <precondition>, the system shall <response>

Event-driven: When <trigger>, the system shall <response>

Optional-feature: Where <feature>, the system shall <response>

Unwanted-behaviour: If <trigger>, then the system shall <response>

Complex: combines While and When clauses

Easy Approach to Requirements Syntax

Six patterns cover happy and unhappy paths

Plain text, no tooling required

Each statement is atomic and testable

In SBCE, the system is always the BC

Every EARS requirement is backed by a test tagged with its id

Easy Approach to Requirements Syntax

R1 — Register for a workshop

- R1.1 (event-driven) — When an attendee submits a registration for a workshop with open seats, the registration shall confirm a seat.
- R1.2 (unwanted) — If the workshop is at capacity, then the registration shall reject the request and offer the waitlist.
- R1.3 (unwanted) — If the email is already confirmed for that workshop, then the registration shall reject the duplicate.

Easy Approach to Requirements Syntax

R2 — Registration window

- R2.1 (state-driven) — While registration is open, the registration shall accept new registrations.
- R2.2 (unwanted) — If the registration deadline has passed, then the registration shall reject the request.

Easy Approach to Requirements Syntax

R3 — Waitlist

- R3.1 (state-driven) — While a workshop is full, the registration shall add new attendees to the waitlist in arrival order.
- R3.2 (complex) — While a workshop is full, when a seat is released, the registration shall promote the first waitlisted attendee to a confirmed seat.

Easy Approach to Requirements Syntax

R4 — Confirmation

- R4.1 (event-driven) — When a seat is confirmed, the registration shall send a confirmation email.
- R4.2 (ubiquitous) — The registration shall record each confirmed attendee's name and email.

Easy Approach to Requirements Syntax

R5 — Discount code

- R5.1 (optional-feature) — Where a valid discount code is provided, the registration shall apply the discount to the fee.
- R5.2 (unwanted) — If the discount code is invalid or expired, then the registration shall keep the full fee and flag the code.

Easy Approach to Requirements Syntax

R6 — Cancellation

- R6.1 (event-driven) — When an attendee cancels a confirmed seat, the registration shall release the seat.
- R6.2 (state-driven) — While the waitlist is non-empty, when a seat is released, the registration shall offer it to the next waitlisted attendee.

spec-kit

spec-kit

Spec-driven development toolkit from GitHub

Specifications generate the implementation, not disposable scaffolding

Seven-step pipeline from principles to code

specify init bootstraps the project and agent commands

/constitution sets project principles and standards

spec-kit

/specify captures requirements and user stories

/clarify resolves underspecified areas by questioning

/plan chooses tech stack and architecture

/tasks breaks the plan into ordered work

/analyze checks cross-artifact consistency

/implement executes tasks, tests first

Artifacts: constitution, spec, plan, tasks, contracts

openspec

openspec

Specs live in a separate openspec/ tree

Splits stable specs from in-flight changes

Each change holds proposal, specs, design, tasks

Lifecycle: explore, propose, apply, archive

openspec

Slash commands drive each stage (/opsx)

CLI: init, validate, archive, list

Completed changes archived, fold into specs

Anchors AI work in artifacts, not chat history

Architecture-agnostic, works with any codebase

kiro 👻

kiro

Agentic IDE from AWS

Spec-driven development from a single prompt

Generates three files: requirements, design, tasks

Acceptance criteria written in EARS notation

Steering files hold project-wide context and standards

kiro

Agent hooks trigger actions on events

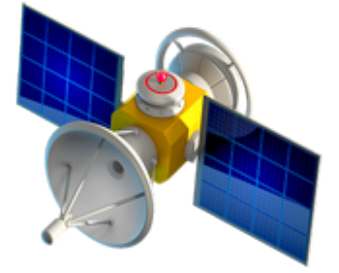
File save can auto-generate tests or docs

Spec mode for structure, vibe mode for chat

Built on Code OSS, supports MCP

Aims to move prototypes toward production code

Spec-Driven BCE ("space")



SPEC-DRIVEN BOUNDARY-CONTROL-ENTITY

Your spec lives in your code. Not in a markdown folder.

SBCE (say "space") is spec-driven development for people who prefer code over markdown. The spec is the boundary package's own doc: one `package-info.java`, co-located with the code it governs, rendered as the published contract by `javadoc`. No parallel `specs/` tree. No drift.

Code over markdown.

[See the idea](#)[Get started](#)

```
/sbce new checkout → /sbce apply checkout
```

...or start from a feature..It asks, then suggests the BCs:

```
/sbce new "let a customer check out a cart"
```

<https://sbce.space>

SBCE / space

Relies on EARS, a documented requirements syntax

Builds on BCE, Jacobson's decades-old robustness pattern

Stores specs in standard package-info.java doc comments

Uses JavaDoc and JEP 467 Markdown, not a custom format

Requirements in plain Markdown, ubiquitous in training data

Targets Java, Jakarta EE, MicroProfile conventions

Follows standard Maven source layout

space

Reuses known test kinds: unit, integration, system

Delegates verification to established runners: JUnit, Playwright, plain java

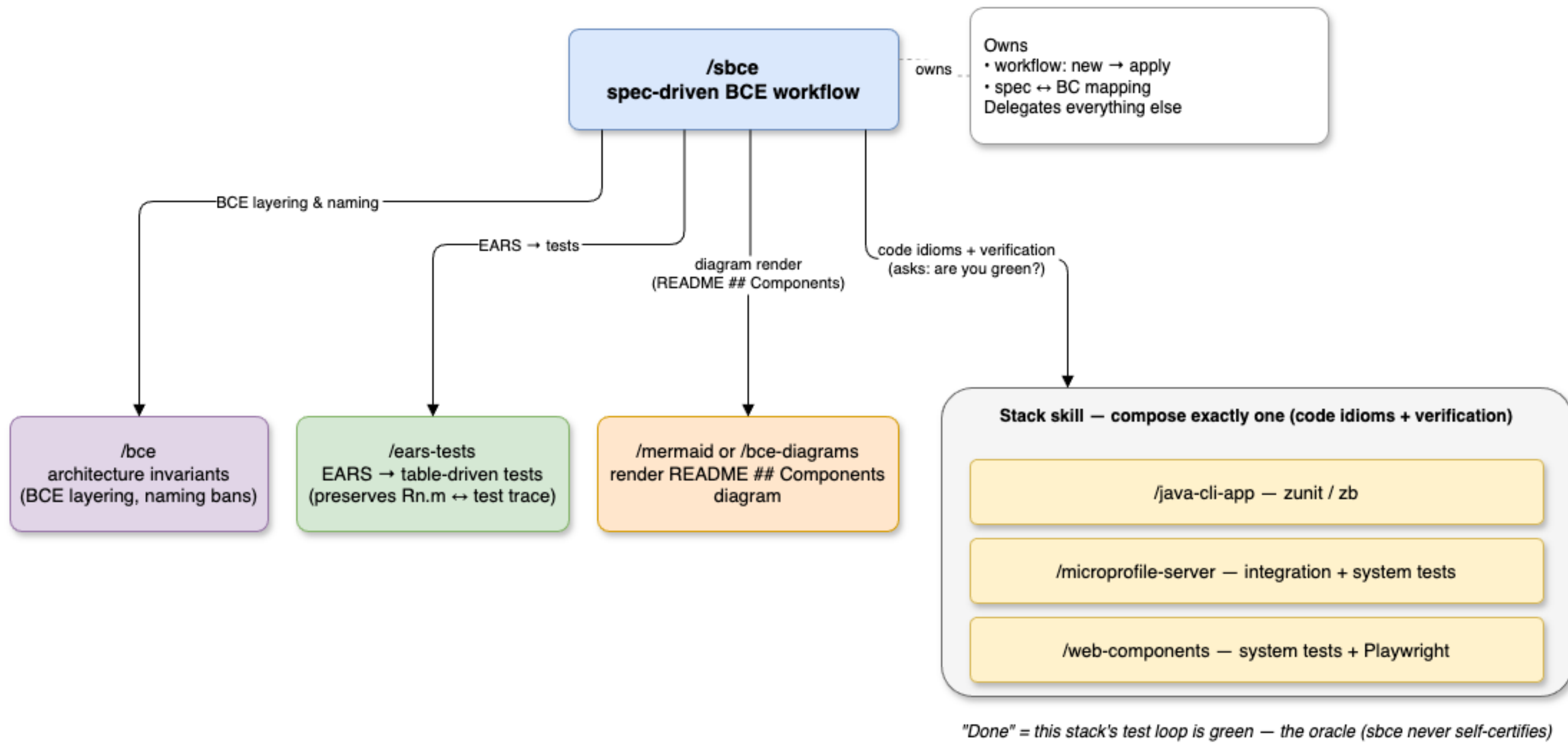
No bespoke DSL, schema, or proprietary spec language

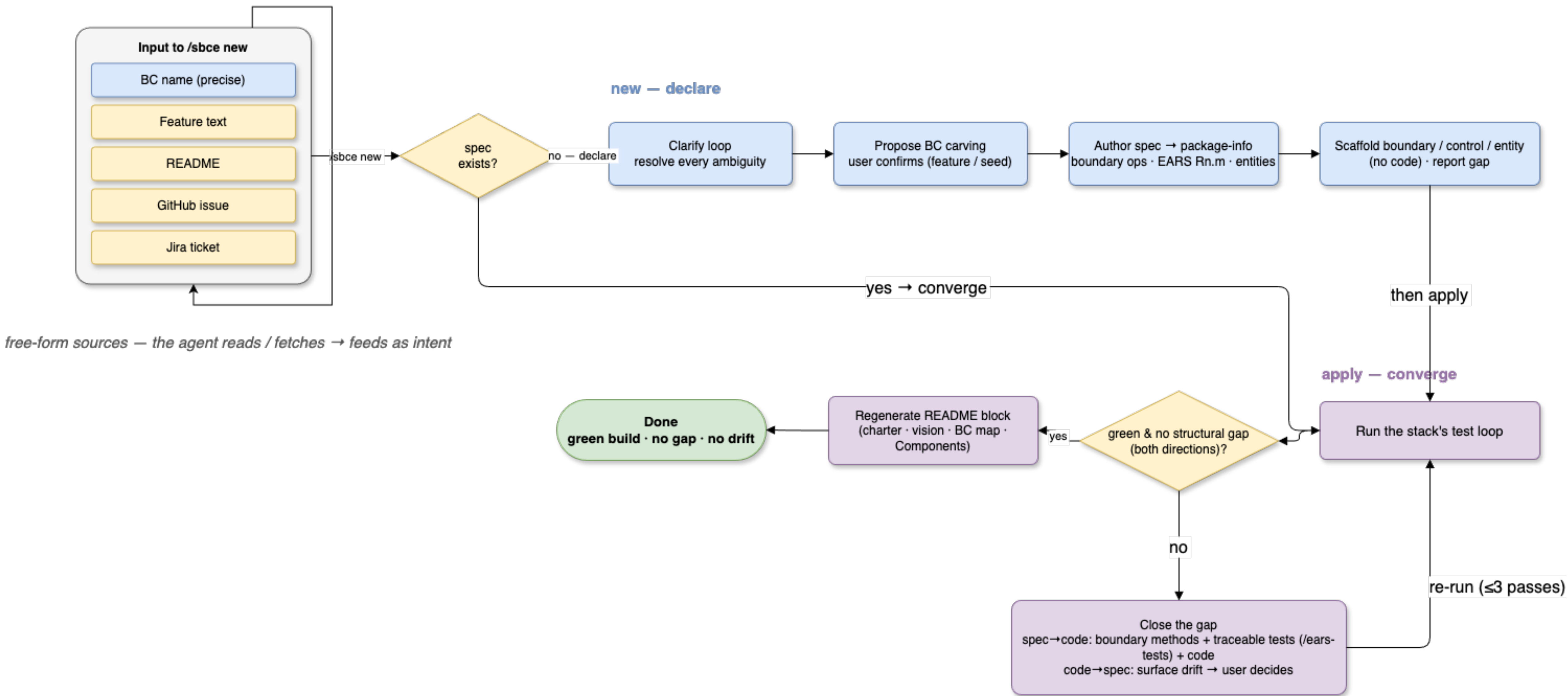
Strong model priors from millions of prior examples

Low hallucination: formats already seen, not invented

Each convention independently documented outside SBCE

Novelty limited to composition, not the parts





space vs. OpenSpec vs. spec-kit

Dimension	/sbce	OpenSpec	Spec Kit
Guiding idea	The spec is the boundary contract, co-located with code	Agree on the change before coding ; specs are a persistent layer	Specs are executable — refined into code through a pipeline
Unit of work	A Business Component (BCE capability)	A change (proposal)	A feature
Where the spec lives	In the code — <code>package-info</code> (<code>javadoc</code>)	Separate <code>openspec/</code> tree (<code>specs/</code> + <code>changes/</code>)	Separate <code>specs/<id>/</code> tree + <code>.specify/</code>
Artifact count	Minimal — one spec + optional system doc/README	Medium — proposal, specs, design, tasks	Maximal — constitution, spec, plan, research, data-model, contracts, tasks
Task list	Computed gap (never stored)	<code>tasks.md</code> checklist	<code>tasks.md</code> (ordered, parallel markers)
Governance	Pluggable skill (<code>/bce</code> + stack) + <code>Sn</code> + README <code>## Conventions</code>	<code>specs/</code> layer + <code>AGENTS.md</code>	<code>constitution.md</code> (one prose doc)
Rationale / "why"	Optional inline <code>_(why:)_</code> on <code>Rn.m</code> (+ git history)	<code>proposal.md</code>	<code>proposal</code> + <code>research.md</code>
Vision	Optional <code>## Vision</code> line	—	Constitution preamble
Lifecycle	<code>new</code> → <code>apply</code> (2 modes)	explore → propose → apply → archive	constitution → specify → clarify → plan → tasks → analyze → implement
"Done" is...	Green test loop + zero gap (binary oracle)	Checklist done + human review	Tasks done + <code>/analyze</code> + review
Traceability	Bijjective, grep-level (<code>Rn.m</code> ↔ test)	Narrative	<code>/analyze</code> cross-artifact check
Architecture	Opinionated — BCE only	Agnostic	Agnostic (stack chosen in <code>/plan</code>)
Tooling	One composable skill, no binary	CLI	CLI + integrations, presets, bundles
Human docs	Generated README projection — charter/vision/BC map + Mermaid <code>## Components</code> (drift-safe); also the inception seed	Hand-written README + generated <code>AGENTS.md</code>	Hand-written <code>quickstart.md</code>

airhacks.live

online, live virtual workshops

Continuous coding, interacting and sharing with [Adam Bien](#)

Live, Virtual Online Workshops, Summer 2026:

Spec-Driven Java Development at LLM Speed, 2 July 2026

...from idea, over specs, to production code

LLM-Assisted Web Components: No Frameworks, No Dependencies, 16 July 2026

...or how to build maintainable web applications with LLMs

Live, Virtual Online Workshops, Winter 2026:

Building Local and Serverless LLM Agents, 15 December 2026

...the same agent, from laptop to cloud

Tickets are also available from: airhacks.eventbrite.com and meetup.com/airhacks

 Prefer in-person, at Munich Airport?

airhacks.university brings hands-on workshops face-to-face. Small group, full day, all of us in one room.



Thank YOU!



airhacks.industries